EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

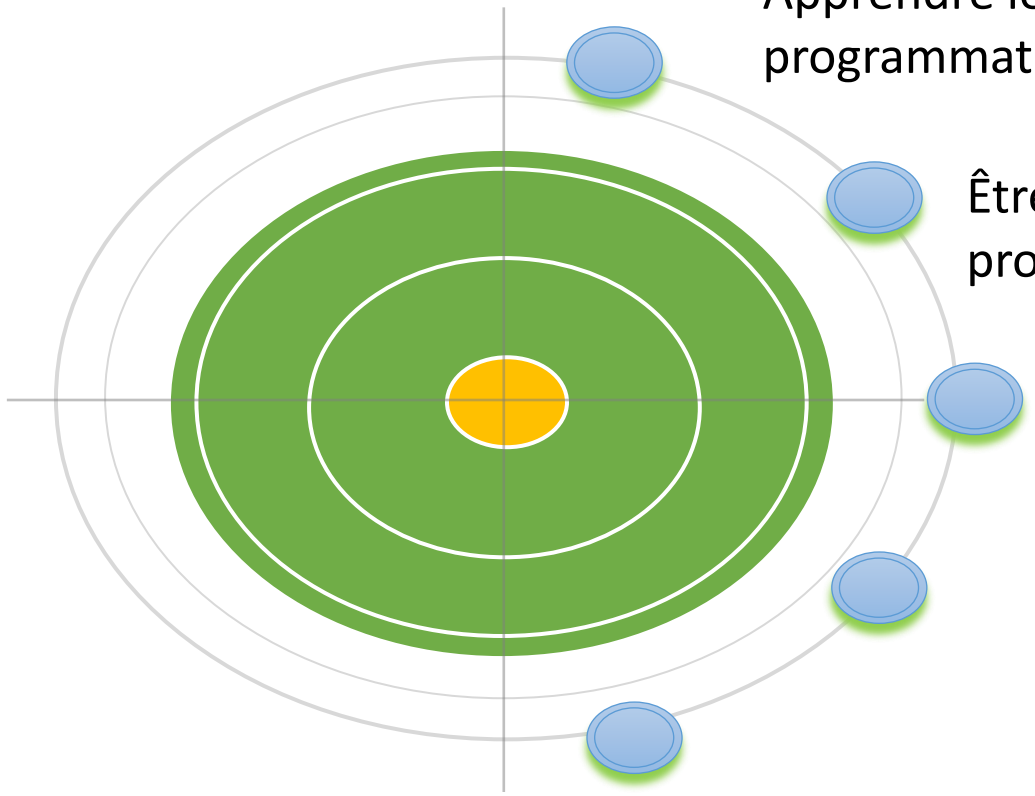
Algorithmique et Programmation

MOHAMMED VI
POLYTECHNIC
UNIVERSITY

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTE

Objectifs

L'étudiant devra :



Apprendre les concepts de base de l'algorithmique et de la programmation

Être capable de mettre en œuvre ces concepts pour analyser des problèmes simples et écrire les programmes correspondants

Références

- Michel Divay : **Algorithmes et structures de données génériques**

- Abdelali Guerid, Pierre Breguet, Henri Röthlisberger : **Algorithmes et structures de données avec C++ et Java**

- Cours Pr. *Nahla Ben Amor* : **Algorithmes et structures de données**, http://isgprog2.ifrance.com/

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Plan

- Introduction à l’algorithmique et à la programmation
- Notion de variable, constantes, affectation, lecture et écriture
- Opérateurs, expressions, type d'une expression.
- Structure d’un programme.
- Fonctions : programmation modulaire, définition, fonctions et procédures en C, passage de paramètres, portée des données, fonctions et tableaux, récursivité.
- Structure Alternatives : conditions, alternative, alternatives imbriquées, choix multiples.
- Structures répétitives : tant que, pour, répéter jusqu’à, imbrications, construction.

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Programme

- Un programme correspond à la description d'une méthode de résolution pour un problème donné.
- Cette description est effectuée par une suite d'instructions d'un langage de programmation
- Ces instructions permettent de traiter et de transformer les données (entrées) du problème à résoudre pour aboutir à des résultats (sorties).
- Un programme n'est pas une solution en soi mais une méthode à suivre pour trouver les solution

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Algorithmique

- algorithme = méthode de résolution
- algorithme vient du nom du célèbre mathématicien

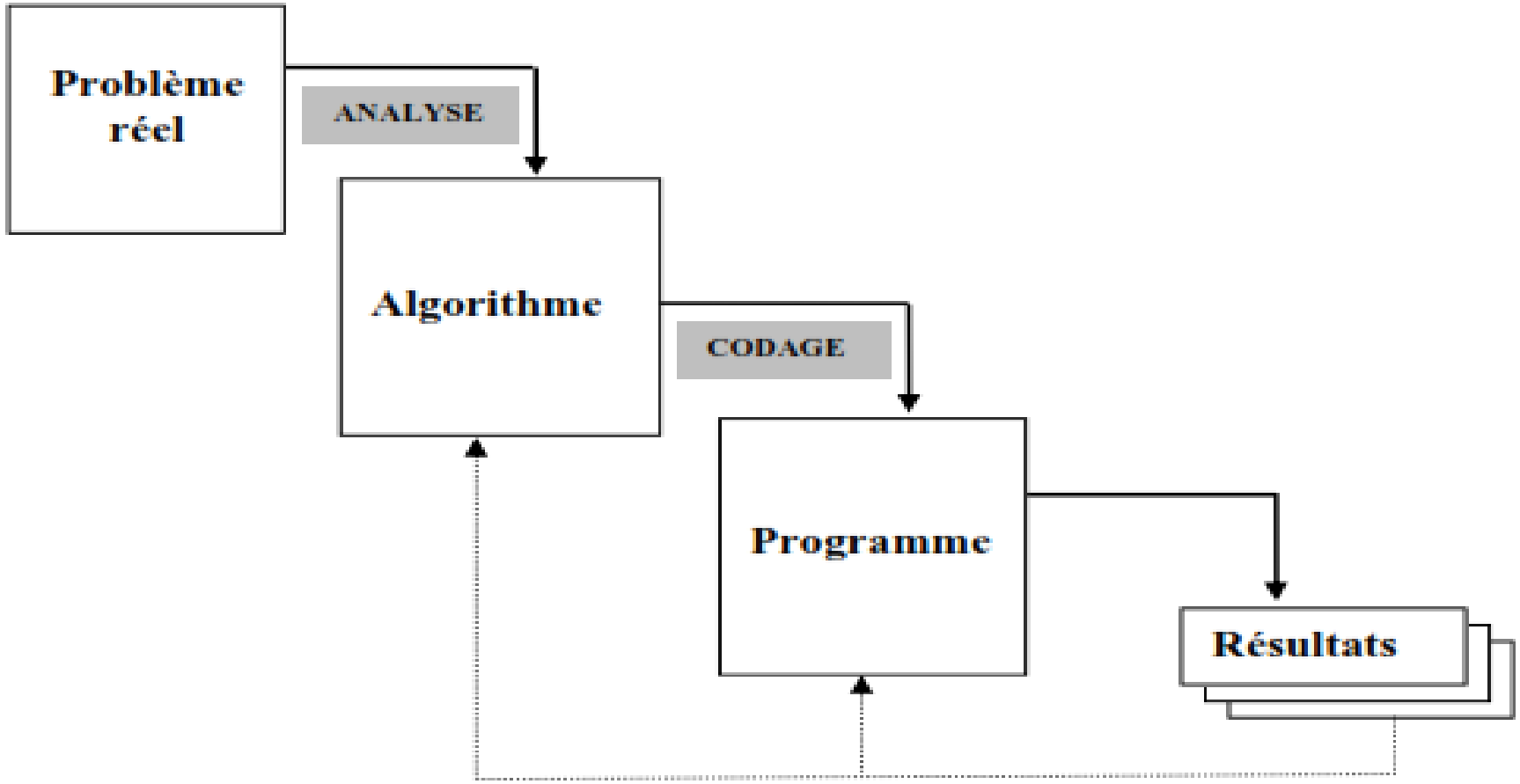
arabe Al Khawarizmi (Abu Ja'far Mohammed Ben Mussa Al-Khwarismi)

  - http ://trucsmaths.free.fr/alkhwarizmi.htm
  - http://publimath.irem.univ-mrs.fr/glossaire/AL016.htm

- L'algorithmique désigne aussi la discipline qui étudie les algorithmes et leurs applications en Informatique.
- Une bonne connaissance de l'algorithmique d'écrire des algorithmes exacts et efficaces.

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

algorithmique



EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Langages informatiques

- Un langage informatique est un code de communication, permettant à un être humain de dialoguer avec une machine en lui soumettant des instructions et en analysant les données matérielles fournies par le système.
- Le langage informatique est l'intermédiaire entre le programmeur et la machine
- Il permet d'écrire des programmes (suite consécutive d'instructions) destinés à effectuer une tache donnée
  - Exemple : un programme de résolution d'une équation du second degré
- Programmation : ensemble des activités orientées vers la conception, la réalisation, le test et la maintenance de programmes

Langages de programmation

- Deux types de langages:
  - Langages procéduraux : Fortran, Cobol, Fortran, Cobol, Pascal, **C**,
  - Langages orientés objets : C++, Java, C#,

- Le choix d'un langage de programmation n'est pas facile, chacun a ses spécificités et correspond mieux à certains types d'utilisations

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Notion d'algorithme

- Un programme informatique permet à l'ordinateur de résoudre un problème :
  - Avant de communiquer à l'ordinateur comment résoudre ce problème, il faut en premier lieu pouvoir le résoudre nous même
- Un algorithme peut se comparer à une recette de cuisine
  - Le résultat c'est comme le plat à cuisiner
  - Les données sont l'analogues des ingrédients de la recette
  - Les règles de transformations se comparent aux directives ou
  - instructions de la recette

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Algorithme informatique

- Un algorithme est une suite d'instructions ayant pour but de résoudre un problème donné. Ces instructions doivent être exécutées de façon automatique par un ordinateur.
- Exemples:
  - préparer une recette de cuisine
  - montrer le chemin à un touriste
  - programmer un magnétoscope
  - etc ...

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Algorithme : exemple

- Pour trouver une valeur approximative de la racine carrée de x
- prendre une approximation initiale arbitraire G
- améliorer cette approximation en calculant la moyenne arithmétique entre G et x/G
- continuer jusqu'à atteindre la précision souhaitée
- Exemple :√x pour x=2
- X = 2 G = 1
- X/G = 2 G = ½ (1+ 2) = 1.5
- X/G = 4/3 G = ½ (3/2 + 4/3) = 17/12 = 1.416666
- X/G = 24/17 G = ½ (17/12 + 24/17) = 577/408 = 1.4142156

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Algorithme et programme

- L’élaboration d’un algorithme précède l’étape de
- programmation
  - Un programme est un algorithme
  - Un langage de programmation est un langage compris par l'ordinateur
- L’élaboration d’un algorithme est une démarche de résolution de problème exigeante
- La rédaction d’un algorithme est un exercice de réflexion qui se fait sur papier
  - L'algorithme est indépendant du langage de programmation
  - Par exemple, on utilisera le même algorithme pour une implantation en Java, ou bien en C ou en Visual Basic
  - L’algorithme est la résolution brute d’un problème informatique

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Propriétés d’un algorithme

- Un algorithme doit:
  - avoir un nombre fini d’étapes,
  - avoir un nombre fini d’opérations par étape,
  - se terminer après un nombre fini d’opérations,
  - fournir un résultat.
- Chaque opération doit être:
  - définie rigoureusement et sans ambiguïté
  - effective, c-à-d réalisable par une machine
  - Le comportement d'un algorithme est déterministe

Représentation d'un algorithme

- Historiquement, deux façons pour représenter un algorithme:
- L'Organigramme: représentation graphique avec des symboles (carrés, losanges, etc.)
  - offre une vue d'ensemble de l'algorithme
  - représentation quasiment abandonnée aujourd'hui
- Le pseudo-code: représentation textuelle avec une série de conventions ressemblant à un langage de programmation
  - plus pratique pour écrire un algorithme
  - représentation largement utilisée

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Notions et Instructions de base

Algorithmique

instructions de base

- Un programme informatique est formé de quatre types d'instructions considérées comme des petites briques de base :
  - l'affectation de variables
  - la lecture et/ou l'écriture
  - les tests
  - les boucles

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Notion de variable

- Une variable sert à stocker la valeur d'une donnée dans un langage de programmation
- Une variable désigne un emplacement mémoire dont le contenu peut changer au cours d'un programme (d'où le nom de variable)
- Chaque emplacement mémoire a un numéro qui permet d'y faire référence de façon unique : c'est l'adresse mémoire de cette cellule
- Règle : La variable doit être déclarée avant d'être utilisée, elle doit être caractérisée par :
  - un nom (Identificateur)
  - un type qui indique l'ensemble des valeurs que peut prendre la variable (entier, réel, booléen, caractère, chaîne de caractères, …) Une valeur

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Dé
Déclaration des variables

- Rappel: toute variable utilisée dans un programme doit avoir fait l’objet d’une déclaration préalable
- En algorithmique, la déclaration de variables est effectuée par la forme suivante :

Variables liste d'identificateurs : type

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Types des variables

- **Type numérique**
  - ENTIER
    - Le type entier désigne l'ensemble des nombres entiers négatifs ou positifs dont les valeurs varient entre -32 768 à 32 767.
  - REEL
    - Le type réel comprend les variables numériques qui ont des valeurs réelles. La plage des valeurs du type réel est :
    - -3,40x10^38 à -1,40x10^45 pour les valeurs négatives
    - 1,40x10^45 à 3,40x10^38 pour les valeurs positives
- **Type caractère**
  - lettres majuscules, minuscules, chiffres, symboles,..
  - Exemples : 'A', 'b', '1', '?', …
- **Type chaîne de caractère**
  - toute suite de caractères
  - Exemples: " " , " Nom, Prénom", "code postale: 1000", **...**
- **Type logique ou booléen**: deux valeurs VRAI ou FAUX

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Déclaration des variables

- Rappel: toute variable utilisée dans un programme doit avoir fait l'objet d'une déclaration préalable
- En algorithmique, la déclaration de variables est effectuée par la forme suivante :

Variables liste d'identificateurs : type

Exemple:

Variables i, j, k : entier

x, y : réel

OK: booléen

Ch1, ch2 : chaîne de caractères

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Constante

- Une constante est une variable dont la valeur ne change pas au cours de l'exécution du programme, elle peut être un nombre, un caractère, ou une chaine de caractères.
- En algorithmique,

Constante identificateur=valeur : type

(par convention, les noms de constantes sont e majuscules)

- Exemple : pour calculer la surface des cercles, la valeur de pi est une constante mais le rayon est une variable.

Constante PI=3.14 : réel, MAXI=32 : entier

- Une constante doit toujours recevoir une valeur dès sa déclaration

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Affectation

- L'affectation consiste à attribuer une valeur à une variable (c'est-à-dire remplir ou modifier le contenu d'une zone mémoire)
- En pseudo-code, l'affectation est notée par le signe ←

Var← e : attribue la valeur de e à la variable Var

- e peut être une valeur, une autre variable ou un expression
- Var et e doivent être de même type ou de types compatibles
- l'affectation ne modifie que ce qui est à gauche de la flèche

(avec i, j, k : entier; x :réel; ok :booléen; ch1,ch2 :chaine de caractères)

Exemples non valides: i ←10.3 OK ←"SMI" j ←x

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Affectation

- Lors d’une affectation, l’expression de droite est évaluée et la valeur trouvée est affectée à la variable de gauche. Ainsi, A←B est différente de B←A
- l'affectation est différente d'une équation mathématique :
  - Les opérations x ← x+1 et x ← x-1 ont un sens en programmation et se nomment respectivement incrémentation et décrémentation.
  - A+1 ← 3 n'est pas possible en langages de programmation et n'est pas équivalente à A ← 2
- Certains langages donnent des valeurs par défaut aux variables déclarées. Pour éviter tout problème il est préférable d'initialiser les variables déclarées.

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Syntaxe générale de l'algorithme

```
Algo exemple
/* La partie déclaration de l'algorithme */
Constantes // les constantes nécessitent une valeur dès leur déclaration
var1←20 : entier
var2←"bonjour!" : chaîne
Variables
// les variables proprement dites
var3, var4 : réels
var5 : chaîne
Début // corps de l'algorithme
/* instructions */
Fin
```

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

la séquence des instructions

- Les opérations d'un algorithme sont habituellement exécutées une à la suite de l'autre, en séquence (de haut en bas et de gauche à droite).
- L'ordre est important.
- On ne peut pas changer cette séquence de façon arbitraire.
- Par exemple, enfiler ses bas puis enfiler ses bottes n’est pas équivalent à enfiler ses bottes puis enfiler ses bas.

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Expressions et opérateurs

- Une expression peut être une valeur, une variable ou une opération constituée de variables reliées par des opérateurs

exemples: 1, b, a*2, a+ 3*b-c, …

- Une expression fournit une valeur unique qui est le résultat de l'opération
- Les opérateurs dépendent du type de l'opération, ils peuvent être :
  - des opérateurs arithmétiques: +, -, *, /, % (modulo), ^(puissance)
  - des opérateurs logiques: NON(!), OU(| |), ET (&&)
  - des opérateurs relationnels: =, <, >, <=, >=
  - des opérateurs sur les chaînes: & (concaténation)
- Une expression est évaluée de gauche à droite mais en tenant compte des priorités des opérateurs.

Expression : remarques

- On ne peut pas additionner un entier et un caractère
- Toutefois dans certains langages on peut utiliser un opérateur avec deux opérandes de types différents, c’est par exemple le cas avec les types arithmétiques (4 + 5.5)
- La signification d’un opérateur peut changer en fonction du type des opérandes
  - l’opérateur + avec des entiers effectue l’addition, 3+6 vaut 9 avec des chaînes de caractères il effectue la concaténation "bonjour" + " tout le monde" vaut "bonjour tout le monde"

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Priorité des opérateurs

- Pour les opérateurs arithmétiques donnés ci-dessus, l'ordre de priorité est le suivant (du plus prioritaire au moins prioritaire) :

  - () : les parenthèses
  - ^ : (élévation à la puissance)
  - * , / (multiplication, division)
  - % (modulo)
  - + , - (addition, soustraction)

- exemple: 9 + 3 * 4 vaut 21
- exemple: (9 + 3) * 4 vaut 48

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Les opérateurs booléens

- Associativité des opérateurs et et ou
  - a et (b et c) = (a et b) et c
- Commutativité des opérateurs et et ou
  - a et b = b et a
  - a ou b = b ou a
- Distributivité des opérateurs et et ou
  - a ou (b et c) = (a ou b) et (a ou c)
  - a et (b ou c) = (a et b) ou (a et c
- Involution (homographie réciproque) : non non a = a
  - Loi de Morgan : non (a ou b) = non a et non b
    non (a et b) = non a ou non b

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Tables de vérité

C1	C2	C1 et C2	C1 ou C2	C1 XOR C2
Vrai	Vrai	Vrai	Vrai	Faux
Vrai	Faux	Faux	Vrai	Vrai
Faux	Vrai	Faux	Vrai	Vrai
Faux	Faux	Faux	Faux	Faux

C1	Non C1
Vrai	Faux
Faux	Vrai

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Les instructions d’entées

- Les instructions de lecture et d'écriture permettent à la machinede communiquer avec l'utilisateur La lecture permet d'entrer des donnés à partir du clavier
- En algo, on note: **lire (var)**
- la machine met la valeur entrée au clavier dans la zone mémoire nommée var
- Remarque: Le programme s'arrête lorsqu'il rencontre une instruction Lire et ne se poursuit qu'après la saisie de l’entrée attendue par le clavier et de la touche Entrée (cette touche signale la fin de l’entrée)
- Conseil: Avant de lire une variable, il est fortement conseillé d’écrire des messages à l’écran, afin de prévenir l’utilisateur de ce qu’il doit frapper

Les instructions d’entées

- L'écriture permet d'afficher des résultats à l'écran (ou de les écrire dans un fichier)
- En , Algo on note: écrire (liste d’expressions)
- la machine affiche les valeurs des expressions décrite dans la liste. Ces instructions peuvent être des variables ayant des valeurs, des nombres ou des commentaires sous forme de chaines de caractères.
- Exemple : écrire(a, b+2, "Message")

Les instructions d'entrées-sorties:

- Les instructions de lecture et d'écriture permettent à la machine de communiquer avec l'utilisateur
- La **lecture** permet d'entrer des <u>donnés</u> à partir du clavier
  - En pseudo-code, on note: **lire (var)**

    la machine met la valeur entrée au clavier
    dans la zone mémoire nommée var
  - <u>Remarque:</u> Le programme s'arrête lorsqu'il rencontre une instruction Lire et ne se poursuit qu'après la frappe d'une valeur au clavier et de la touche Entrée

Les instructions d'entrées-sorties

- **L'écriture** permet d'afficher des <u>résultats</u> à l'écran (ou de les écrire dans un fichier)
  - En pseudo-code, on note: **écrire (var)**
    la machine affiche le contenu de la zone mémoire var
  - <u>Conseil:</u> Avant de lire une variable, il est fortement conseillé d'écrire des messages à l'écran, afin de prévenir l'utilisateur de ce qu'il doit frapper

Exemple (lecture et écriture)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui calcule et affiche le double de ce nombre

Algorithme Calcul_double

variables A, B : **entier**

Début

écrire("entrer le nombre ")

lire(A)

B ← 2*A

écrire("le double de ", A, "est :", B)

Fin

Exercice (lecture et écriture)

Ecrire un algorithme qui vous demande de saisir votre nom puis votre prénom et qui affiche ensuite votre nom complet

Algorithme AffichageNomComplet

variables Nom, Prenom, Nom_Complet : **chaîne de caractères**

Début

écrire("entrez votre nom")

lire(Nom)

écrire("entrez votre prénom")

lire(Prenom)

Nom_Complet ← Nom & Prenom

écrire("Votre nom complet est : ", Nom_Complet)

Fin

Tests: instructions conditionnelles (1)

- Les instructions conditionnelles servent à n'exécuter une instruction ou une séquence d'instructions que si une condition est vérifiée

- On utilisera la forme suivante: **Si** condition **alors**

  instruction ou suite d'instructions1

  Sinon

  instruction ou suite d'instructions2

  Finsi

  - la condition ne peut être que vraie ou fausse
  - si la condition est vraie, se sont les instructions1 qui seront exécutées
  - si la condition est fausse, se sont les instructions2 qui seront exécutées
  - la condition peut être une condition simple ou une condition composée de plusieurs conditions

Tests: instructions conditionnelles (2)

- La partie Sinon n'est pas obligatoire, quand elle n'existe pas et que la condition est fausse, aucun traitement n'est réalisé
  - On utilisera dans ce cas la forme simplifiée suivante:

Si condition **alors**
    instruction ou suite d'instructions1
Finsi

Exemple (Si…Alors…Sinon)

```
Algorithme AffichageValeurAbsolue (version1)
Variable x : réel
Début
        Ecrire (" Entrez un réel : “)
        Lire (x)
        Si (x < 0) alors
                Ecrire ("la valeur absolue de ", x, "est:",-x)
        Sinon
                Ecrire ("la valeur absolue de ", x, "est:",x)
        Finsi
Fin
```

Exemple (Si…Alors)

Algorithme AffichageValeurAbsolue (version2)

Variable x,y : **réel**

Début

Ecrire (" Entrez un réel : “)

Lire (x)

y← x

Si (x < 0) **alors**

y ← -x

Finsi

Ecrire ("la valeur absolue de ", x, "est:",y)

Fin

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Exercice (tests)

Ecrire un algorithme qui demande un nombre entier à l'utilisateur, puis qui teste et affiche s'il est divisible par 3

```
Algorithme Divsible_par3
Variable n : entier
Début
        Ecrire " Entrez un entier : "
        Lire (n)
        Si (n%3=0) alors
                Ecrire (n," est divisible par 3")
        Sinon
                Ecrire (n," n'est pas divisible par 3")
        Finsi
Fin
```

Conditions composées

- Une condition composée est une condition formée de plusieurs conditions simples reliées par des opérateurs logiques:

  ET, OU, OU exclusif (XOR) et NON

- Exemples :
  - x compris entre 2 et 6 : (x > 2) ET (x < 6)
  - n divisible par 3 ou par 2 : (n%3=0) OU (n%2=0)
  - deux valeurs et deux seulement sont identiques parmi a, b et c :
    (a=b) XOR (a=c) XOR (b=c)

- L'évaluation d'une condition composée se fait selon des règles présentées généralement dans ce qu'on appelle <u>tables de vérité</u>

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Tables de vérité

C1	C2	**C1 ET C2**
VRAI	VRAI	**VRAI**
VRAI	FAUX	**FAUX**
FAUX	VRAI	**FAUX**
FAUX	FAUX	**FAUX**

C1	C2	**C1 OU C2**
VRAI	VRAI	**VRAI**
VRAI	FAUX	**VRAI**
FAUX	VRAI	**VRAI**
FAUX	FAUX	**FAUX**

C1	C2	**C1 XOR C2**
VRAI	VRAI	**FAUX**
VRAI	FAUX	**VRAI**
FAUX	VRAI	**VRAI**
FAUX	FAUX	**FAUX**

C1	**NON C1**
VRAI	**FAUX**
FAUX	**VRAI**

Tests imbriqués

- Les tests peuvent avoir un degré quelconque d'imbrications

```
Si condition1 alors
        Si condition2 alors
                instructionsA
        Sinon
                instructionsB
        Finsi
Sinon
        Si condition3 alors
                instructionsC
        Finsi
Finsi
```

Tests imbriqués: exemple (version 1)

```
Variable n : entier
Début
        Ecrire ("entrez un nombre : ")
        Lire (n)
        Si (n < 0) alors
                Ecrire ("Ce nombre est négatif")
        Sinon
                Si (n = 0) alors
                        Ecrire ("Ce nombre est nul")
                Sinon
                        Ecrire ("Ce nombre est positif")
                Finsi
        Finsi
Fin
```

Tests imbriqués: exemple (version 2)

```
Variable n : entier
 Début
        Ecrire ("entrez un nombre : ")
        Lire (n)

        Si (n < 0) alors        Ecrire ("Ce nombre est négatif")
        Finsi

        Si (n = 0) alors        Ecrire ("Ce nombre est nul")

        Finsi
        Si (n > 0) alors        Ecrire ("Ce nombre est positif")

        Finsi

 Fin
```

Remarque : dans la version 2 on fait trois tests systématiquement alors que dans la version 1, si le nombre est négatif on ne fait qu'un seul test

Conseil : utiliser les tests imbriqués pour limiter le nombre de tests et placer d'abord les conditions les plus probables

Tests imbriqués: exercice

Le prix de photocopies dans une reprographie varie selon le nombre demandé: 0,5 DH la copie pour un nombre de copies inférieur à 10, 0,4DH pour un nombre compris entre 10 et 20 et 0,3DH au-delà.

Ecrivez un algorithme qui demande à l'utilisateur le nombre de photocopies effectuées, qui calcule et affiche le prix à payer

Tests imbriqués: corrigé de l'exercice

```
Variables copies : entier
          prix : réel
Début
    Ecrire ("Nombre de photocopies : ")

    Lire (copies)
    Si (copies < 10) Alors
       prix ← copies*0.5

    Sinon Si (copies) < 20
              prix ← copies*0.4
        Sinon
              prix ← copies*0.3
        Finsi

    Finsi
    Ecrire (“Le prix à payer est : ”, prix)

Fin
```

Instructions itératives: les boucles

- Les boucles servent à répéter l'exécution d'un groupe d'instructions un certain nombre de fois

- On distingue trois sortes de boucles en langages de programmation :
  - Les **boucles tant que** : on y répète des instructions tant qu'une certaine condition est réalisée
  - Les **boucles jusqu'à** : on y répète des instructions jusqu'à ce qu'une certaine condition soit réalisée
  - Les **boucles pour** ou avec compteur : on y répète des instructions en faisant évoluer un compteur (variable particulière) entre une valeur initiale et une valeur finale

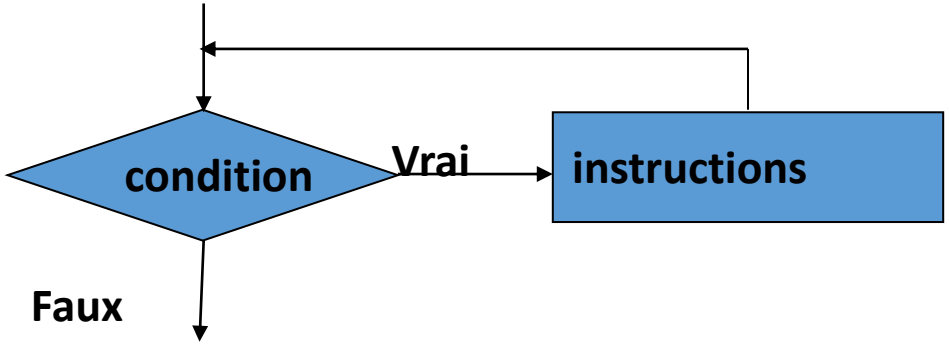
(Dans ce cours, on va s'intéresser essentiellement aux boucles *Tant que* et boucles *Pour* qui sont plus utilisées et qui sont définies en Maple)

Les boucles Tant que

TantQue (condition)

instructions

FinTantQue



- la condition (dite condition de contrôle de la boucle) est évaluée avant chaque itération
- si la condition est vraie, on exécute instructions (corps de la boucle), puis, on retourne tester la condition. Si elle est encore vraie, on répète l'exécution, …
- si la condition est fausse, on sort de la boucle et on exécute l'instruction qui est après FinTantQue

Les boucles Tant que : remarques

- Le nombre d'itérations dans une boucle TantQue n'est pas connu au moment d'entrée dans la boucle. Il dépend de l'évolution de la valeur de condition

- Une des instructions du corps de la boucle doit absolument changer la valeur de condition de vrai à faux (après un certain nombre d'itérations), sinon le programme tourne indéfiniment

Attention aux boucles infinies

- Exemple de boucle infinie : $\Longrightarrow$

```
i ← 2
TantQue   (i > 0)
i ←  i+1        (attention aux erreurs de frappe : + au lieu de -)
FinTantQue
```

Boucle Tant que : exemple1

Contrôle de saisie d'une lettre majuscule jusqu'à ce que le caractère entré soit valable

```
Variable C : caractère
Debut
        Ecrire (" Entrez une lettre majuscule ")
        Lire (C)
        TantQue (C < 'A' ou C > 'Z')
                Ecrire ("Saisie erronée. Recommencez")
                Lire (C)
        FinTantQue
        Ecrire ("Saisie valable")
Fin
```

Boucle Tant que : exemple2

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100

<u>version 1</u>

```
Variables som, i : entier
Debut
        i ← 0
        som← 0
        TantQue (som <=100)
                i ←  i+1
                som ←  som+i
        FinTantQue
        Ecrire (" La valeur cherchée est N=  ", i)
Fin
```

Boucle Tant que : exemple2 (version2)

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100

<u>version 2:</u> attention à l'ordre des instructions et aux valeurs initiales

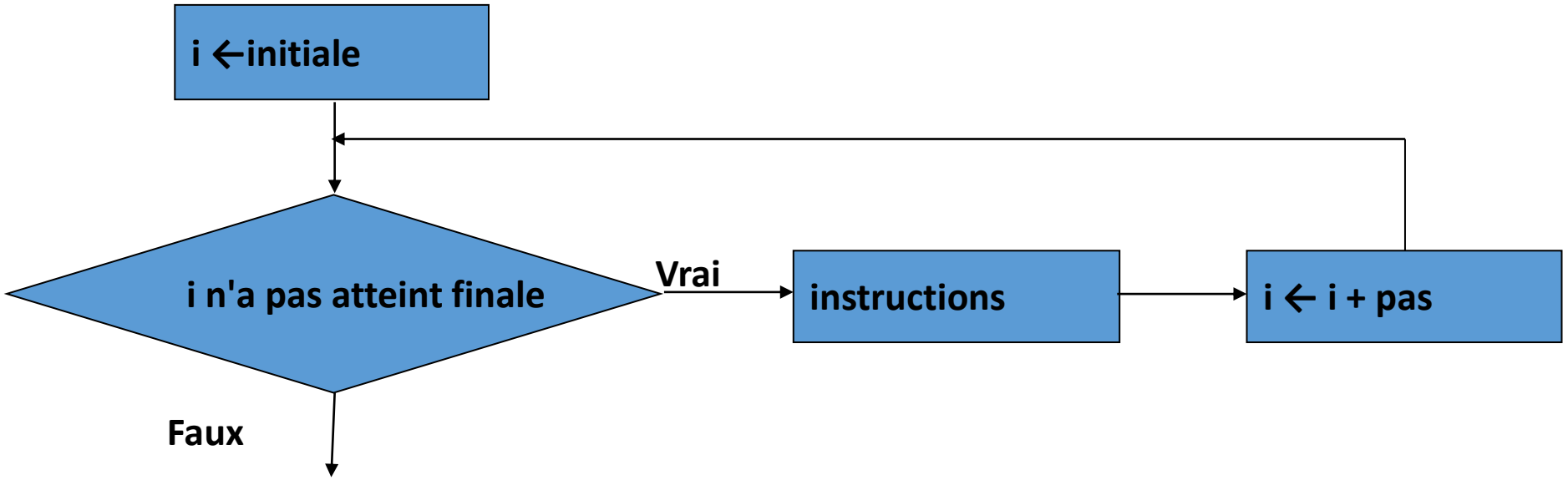
```
Variables som, i : entier
Debut
        som ← 0
        i ← 1
        TantQue (som <=100)
                        som ← som + i
                i ←  i+1
        FinTantQue
        Ecrire (" La valeur cherchée est N=  ", i-1)
Fin
```

Les boucles Pour

Pour compteur **allant de** initiale **à** finale par **pas** valeur du pas

instructions

FinPour



Les boucles Pour

- Remarque : le nombre d'itérations dans une boucle Pour est connu avant le début de la boucle

- **Compteur** est une variable de type entier (ou caractère). Elle doit être déclarée

- **Pas** est un entier qui peut être positif ou négatif. **Pas** peut ne pas être mentionné, car par défaut sa valeur est égal à 1. Dans ce cas, le nombre d'itérations est égal à finale - initiale+ 1

- **Initiale et finale** peuvent être des valeurs, des variables définies avant le début de la boucle ou des expressions de même type que compteur

Déroulement des boucles Pour

1) La valeur initiale est affectée à la variable compteur

2) On compare la valeur du compteur et la valeur de finale :

   a) Si la valeur du compteur est > à la valeur finale dans le cas d'un pas positif (ou si compteur est < à finale pour un pas négatif), on sort de la boucle et on continue avec l'instruction qui suit FinPour

   b) Si compteur est <= à finale dans le cas d'un pas positif (ou si compteur est >= à finale pour un pas négatif), instructions seront exécutées

      i. Ensuite, la valeur de compteur est incrémentée de la valeur du pas si pas est positif (ou décrémenté si pas est négatif)

      ii. On recommence l'étape 2 : La comparaison entre compteur et finale est de nouveau effectuée, et ainsi de suite …

Boucle Pour : exemple1

Calcul de x à la puissance n où x est un réel non nul et n un entier positif ou nul

```
Variables x, puiss : réel
                n, i : entier
Debut
        Ecrire (" Entrez la valeur de x ")
        Lire (x)
        Ecrire (" Entrez la valeur de n ")
        Lire (n)

        puiss ← 1
        Pour i allant de 1 à n
                puiss← puiss*x
        FinPour
        Ecrire (x, " à la puissance  ", n, "  est égal à  ", puiss)
Fin
```

Boucle Pour : exemple1 (version 2)

Calcul de x à la puissance n où x est un réel non nul et n un entier positif ou nul (**version 2 avec un pas négatif**)

```
Variables x, puiss : réel
                n, i : entier
Debut
        Ecrire (" Entrez respectivement les valeurs de x et n ")
        Lire (x, n)
        puiss ← 1
        Pour i allant de n à 1 par pas -1
                puiss← puiss*x
        FinPour
        Ecrire (x, " à la puissance  ", n, "  est égal à  ", puiss)
Fin
```

Boucle Pour : remarque

- Il faut éviter de modifier la valeur du compteur (et de finale) à l'intérieur de la boucle. En effet, une telle action :
  - perturbe le nombre d'itérations prévu par la boucle Pour
  - rend difficile la lecture de l'algorithme
  - présente le risque d'aboutir à une boucle infinie

Exemple : **Pour** i **allant de** 1 **à** 5

i ← i -1

écrire(" i = ", i)

Finpour

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Lien entre Pour et TantQue

La boucle Pour est un cas particulier de Tant Que (cas où le nombre d'itérations est connu et fixé) . Tout ce qu'on peut écrire avec Pour peut être remplacé avec TantQue (la réciproque est fausse)

Pour compteur **allant de** initiale **à** finale par **pas** valeur du pas

instructions

FinPour

peut être remplacé par :

(cas d'un pas positif)

compteur ← initiale
TantQue compteur <= finale
instructions
compteur ← compteur+pas
FinTantQue

Lien entre Pour et TantQue: exemple

Calcul de x à la puissance n où x est un réel non nul et n un entier positif ou nul (**version avec TantQue**)

```
Variables x, puiss : réel
          n, i : entier
Debut
    Ecrire (" Entrez la valeur de x ")
    Lire (x)
    Ecrire (" Entrez la valeur de n ")
    Lire (n)

    puiss ← 1
    i ← 1
    TantQue (i<=n)
        puiss← puiss*x
        i ← i+1
    FinTantQue
    Ecrire (x, " à la puissance  ", n, "  est égal à  ", puiss)
Fin
```

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Boucles imbriquées

- Les instructions d'une boucle peuvent être des instructions itératives. Dans ce cas, on aboutit à des **boucles imbriquées**

- **Exemple:**

```
Pour i allant de 1 à 5
        Pour j allant de 1 à i
                écrire("O")
        FinPour
        écrire("X")
FinPour
```

Exécution

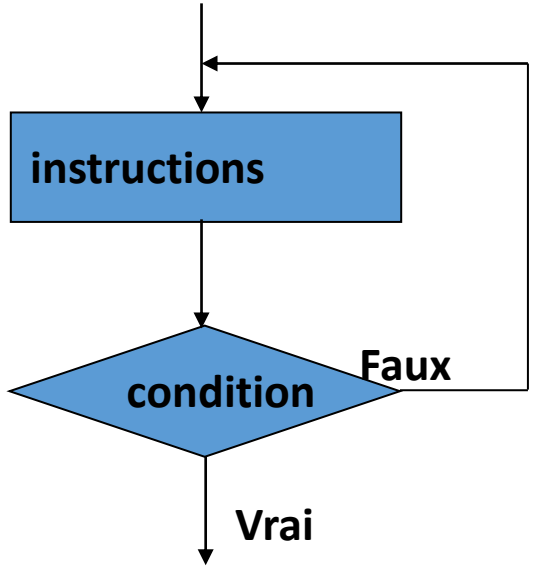
OX
OOX
OOOX
OOOOX
OOO(

Les boucles **Répéter … jusqu'à …**

Répéter

instructions

Jusqu'à condition



- Condition est évaluée après chaque itération
- les instructions entre *Répéter* et *jusqu'à* sont exécutées <u>au moins une fois</u> et leur exécution est répétée jusqu'à ce que condition soit vrai (tant qu'elle est fausse)

Boucle **Répéter jusqu'à** : exemple

Un algorithme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100
(version avec répéter jusqu'à)

```
Variables som, i : entier
Debut
        som ← 0
        i ← 0
        Répéter
                        i ←  i+1
                som ←  som+i
        Jusqu'à ( som > 100)
        Ecrire (" La valeur cherchée est N=  ", i)
Fin
```

Choix d'un type de boucle

- Si on peut déterminer le nombre d'itérations avant l'exécution de la boucle, il est plus naturel d'utiliser *la boucle Pour*

- S'il n'est pas possible de connaître le nombre d'itérations avant l'exécution de la boucle, on fera appel à l'une des *boucles TantQue ou répéter jusqu'à*

- Pour le choix entre *TantQue* et *jusqu'à :*

  - Si on doit tester la condition de contrôle avant de commencer les instructions de la boucle, on utilisera *TantQue*

  - Si la valeur de la condition de contrôle dépend d'une première exécution des instructions de la boucle, on utilisera *répéter jusqu'à*

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE